

Lecture 15: Binary Search on Answer Space (Advanced Paradigms)

1. The Core Paradigm: "Binary Search on Answer"

In previous lectures, Binary Search was strictly used to find the *index* of a specific element inside a monotonic array. In advanced competitive programming, we flip this paradigm: we apply Binary Search directly on the **theoretical range of possible answers**.

- **The Identification Pattern:** You can apply this paradigm if a problem asks you to "**Minimize the Maximum**" or "**Maximize the Minimum**" of a certain configuration, AND if a valid answer at x implies all answers greater than x (or less than x) are also valid (forming a True/False monotonic boundary).
- **The Search Space:** The lowest possible answer (`start`) and the highest possible theoretical answer (`end`).
- **The Predicate Function (`isPossible`):** A custom boolean function that simulates the problem constraints. Given a candidate answer `mid`, it returns `True` if the candidate is mathematically possible, and `False` otherwise.

Problem-Solving Deconstruction

Problem 1: Book Allocation Problem / Ayush Gives Ninja Test

1. Problem Statement & Constraints: Given an array representing pages (or time) of M books and N students (or days), allocate contiguous books to students such that the maximum number of pages assigned to a single student is **minimized**. Every student must get at least one book.

2. Core Intuition: * **Minimum Possible Answer (`start`):** 0.

- **Maximum Possible Answer (`end`):** The sum of all pages in the array (the worst-case scenario where 1 student reads absolutely everything).
- **Predicate Logic:** If we guess that `mid` is the maximum allowed pages per student, we linearly distribute books. If adding a book exceeds `mid`, we hand it to the next student. If the required students exceed N , or a single book is larger than `mid`, the guess `mid` is invalid (`False`).

3. Algorithmic Steps:

1. Initialize `s = 0` and compute `sum` of the array. Set `e = sum`.
2. Calculate `mid = s + (e - s) / 2`.
3. Evaluate `isPossibleSolution(mid)`:

- Iterate through books. Accumulate `timeCount`.
 - If `timeCount + current_book > mid`, allocate to a new student (`dayCount++`). Reset `timeCount = current_book`.
 - If `dayCount > N` or `current_book > mid`, return `false`.
4. **Valid Guess (True):** Record `ans = mid`. To minimize the maximum, restrict the search space to the left: `e = mid - 1`.
 5. **Invalid Guess (False):** The limit was too strict. Relax the limit by moving right: `s = mid + 1`.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(M \log(\sum \text{pages}))$	The search space is bounded by $\sum \text{pages}$. Each of the $\log(\sum \text{pages})$ binary search steps runs an $O(M)$ linear loop to validate the allocation.
Space Complexity	$O(1)$	Validation uses constant auxiliary trackers.

C++

```
#include <bits/stdc++.h>
bool isPossibleSolution(int n, int m, vector<int> time, long long mid) {
    int dayCount = 1;
    long long timeCount = 0;

    for (int i=0; i<m; i++) {
        if (timeCount + time[i] <= mid) {
            timeCount += time[i];
        }
        else {
            dayCount++;
            if (dayCount>n || time[i]>mid) {
                return false;
            }
            timeCount = time[i];
        }
    }
    return true;
}

long long ayushGivesNinjatest(int n, int m, vector<int> time) {
    long long s = 0;
    long long sum = 0;
    for (int i=0; i<m; i++) sum += time[i];

    long long e = sum;
    long long mid = s + (e-s)/2;
```

```

long long ans = -1;

while (s<=e) {
    if (isPossibleSolution(n, m, time, mid)) {
        ans = mid;
        e = mid - 1; // Seek a smaller valid maximum
    }
    else {
        s = mid + 1;
    }
    mid = s + (e-s)/2;
}
return ans;
}

```

(Note: The **Painter's Partition Problem** uses exactly identical logic. Books $\rightarrow$ Boards, Students $\rightarrow$ Painters. The code structure remains mathematically identical).

Problem 2: Aggressive Cows

1. Problem Statement & Constraints: Given an array of stall coordinates and K aggressive cows, place the cows in the stalls such that the **minimum distance** between any two cows is **maximized**.

2. Core Intuition: * **Sorting Prerequisite:** The stalls must be sorted sequentially on a 1D line to accurately measure contiguous gaps.

- **Maximum Possible Answer (end):** The largest possible distance is placing cows at the absolute extremes (e.g., $end = \max(stalls)$).
- **Predicate Logic:** Try placing Cow 1 at `stalls[0]`. Iterate through remaining stalls. If the gap between the current stall and the last placed cow is $\geq mid$, place the next cow. If we successfully place K cows, `mid` is a valid minimum distance.

3. Algorithmic Steps:

1. Sort the `stalls` array.
2. Set `s = 0` and `e = max_element(stalls)`.
3. Evaluate `isPossibleSolution(mid)`:
 - Keep a `cowCount = 1` and `lastPos = stalls[0]`.
 - Loop `i` from 0 to `size`. If `stalls[i] - lastPos >= mid`, place cow (`cowCount++`) and update `lastPos = stalls[i]`.
 - If `cowCount == K`, return `true`.
4. **Valid Guess (True):** Record `ans = mid`. Since we want to *maximize* this distance, aggressively push the boundary to the right: `s = mid + 1`.

5. **Invalid Guess (False):** The cows couldn't fit. Decrease the target distance: `e = mid - 1`.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N \log N + N \log(\max(\text{stalls})))$	$O(N \log N)$ to initially sort. The binary search executes $\log(\max)$ times, evaluating an $O(N)$ placement function.
Space Complexity	$O(1)$	Modified in-place via variables.

C++

```
#include<algorithm>

bool isPossibleSolution(vector<int> &stalls, int k, int mid) {
    int cowCount = 1;
    int lastPos = stalls[0];

    for (int i=0; i<stalls.size(); i++) {
        if (stalls[i]-lastPos >= mid) {
            cowCount++;
            if (cowCount == k) return true;
            lastPos = stalls[i];
        }
    }
    return false;
}

int aggressiveCows(vector<int> &stalls, int k) {
    sort(stalls.begin(), stalls.end());
    int s = 0;
    int e = *max_element(stalls.begin(), stalls.end());
    int ans = -1;
    int mid = s + (e-s)/2;

    while (s<=e) {
        if (isPossibleSolution(stalls, k, mid)) {
            ans = mid;
            s = mid + 1; // Seek a larger valid minimum gap
        }
        else {
            e = mid - 1;
        }
        mid = s + (e-s)/2;
    }
}
```

```

    return ans;
}

```

Problem 3: Cooking Ninjas (Prata)

1. Problem Statement & Constraints: You have an array representing the rank of various cooks. A cook with rank R takes $1R$ time for the 1st dish, $2R$ time for the 2nd, $3R$ time for the 3rd, etc. Find the **minimum total time** required to cook exactly M dishes in parallel.

2. Core Intuition: * **Maximum Possible Answer (end):** The worst-case scenario occurs if only one cook (e.g., `rank[0]`) makes all M dishes alone. We simulate the arithmetic progression sum for this boundary: $R \times (1 + 2 + \dots + M)$.

- **Predicate Logic:** For a given guess `mid` (total time limit), iterate through every single cook. Check how many consecutive dishes they can make before their personal time accumulator exceeds `mid`. Sum the dishes made by all cooks. If the total dishes $\geq M$, the time limit `mid` is valid.

3. Algorithmic Steps:

1. Establish `start = 0`. Compute upper bound `maxi` by simulating the first cook making all M dishes. `end = maxi`.
2. Evaluate `isPossible(mid)`:
 - Loop through each `cook`. Reset `time = 0`, `R = 1`.
 - Loop while `(time + R * rank[cook] <= mid)`: Increment `numOfDish`, add to `time`, and increment `R`.
 - If at any point `numOfDish == m`, return `true`.
3. **Valid Guess (True):** Record `ans = mid`. We want to minimize the time, so squeeze the right boundary: `end = mid - 1`.
4. **Invalid Guess (False):** Not enough time to cook M dishes. Increase time: `start = mid + 1`.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\text{Cooks} \times \sqrt{\text{max_time}} \times \log(\text{max_time}))$	Binary search runs $\log(\text{max})$ times. The inner validation loops over C cooks, and since dish times grow via arithmetic progression, a cook makes $\approx \sqrt{\text{mid}/R}$ dishes.

Metric	Complexity	Justification
Space Complexity	$O(1)$	No external arrays generated.

C++

```
#include <bits/stdc++.h>

bool isPossible(const vector<int> &rank, int m, int mid) {
    int numOfDish = 0;
    int time = 0, R = 1;

    for (int cook = 0; cook < rank.size(); cook++) {
        // While the current cook has time to make the next dish
        while (time + R * rank[cook] <= mid) {
            if (numOfDish == m) return true;
            numOfDish++;
            time += R * rank[cook];
            R++;
        }
        time = 0; // Reset for next cook
        R = 1;
    }
    return numOfDish >= m;
}

int minCookTime(const vector<int> &rank, int m) {
    int start = 0;
    int maxi = 0;

    // Establishing worst-case upper boundary using first cook
    for (int i = 1; i <= m; i++) {
        maxi = maxi + (i * rank[0]);
    }

    int end = maxi;
    int mid, ans = -1;

    while (start <= end) {
        mid = start + (end - start) / 2;
        if (isPossible(rank, m, mid)) {
            ans = mid;
            end = mid - 1; // Optimize for a smaller valid time
        }
        else {
            start = mid + 1;
        }
    }
}
```

```
return ans;
```

```
}
```